COMP3028 / SOFTWARE CONSTRUCTION

Optimising Server Management

Lecture 10

Beginner explanations and guided experiments

The launch worked. Why are users still waiting?

A campus event opens registration at 09:00. The site worked yesterday with five testers. Today, 500 students arrive together.

Opening problem

The process is running, but registration takes 12 seconds and some requests fail. Is the service healthy?

Discuss: What would you measure first? Would buying a larger server necessarily solve the problem?

What you will be able to do

- Explain the operational work that continues after deployment.
- Calculate availability and distinguish an SLI, SLO, and SLA.
- Measure latency, errors, resource use, and throughput.
- Diagnose blocking, test caching, and compare one versus two workers.
- Explain where state belongs when an application scales.

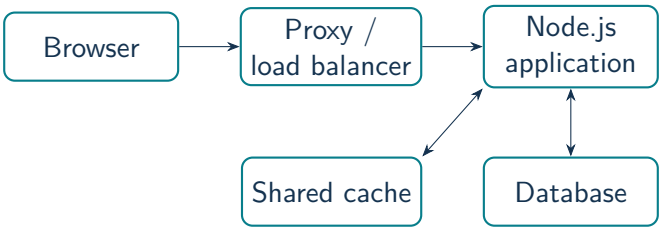
The learning route

- Server management: keep the application usable after launch.
- Service commitments: define what good service means.
- Monitoring: collect evidence about user experience and resources.
- Optimisation and scaling: change one thing and measure again.
- Hands-on practice, incident response, and project evidence.

Server management is ongoing engineering

- A server may mean a physical machine, a virtual machine, or an application process. These layers fail in different ways.
- Management includes patching, configuration, monitoring, backups, capacity planning, and recovery.
- A successful deployment starts the operational phase. It does not prove that users can complete their tasks.

Following a request through the system



Delay can occur at any layer. A healthy application process cannot guarantee a healthy database or network.

A practical maintenance routine

- Before a change: record the current version, test the change, and prepare rollback.
- During a change: watch user-facing errors and latency.
- After a change: verify important journeys and compare against the baseline.
- Periodically: test restoring backups, review permissions, and check capacity. An untested backup may not support recovery.

Configuration changes can undo good code

- Bind services to the intended interfaces and expose only required ports.
- Replace default credentials and keep secrets outside source control.
- Set appropriate permissions, request limits, and timeouts.
- Review reverse-proxy settings before trusting forwarded client addresses.
- Apply updates through a tested release process; newer does not automatically mean compatible.

Performance, availability, and correctness differ

- Performance asks how quickly an operation completes under a stated load.
- Availability asks whether the defined service is usable during a measurement window.
- Correctness asks whether the result is right. A fast HTTP 200 response with wrong data is still a failure.
- Define success from the user journey: a completed registration matters more than a running process.

SLI, SLO, and SLA

Term	Meaning	Classroom example
SLI	Measured indicator	99.94% of eligible requests succeeded.
SLO	Target for that indicator	At least 99.9% succeed over 30 days.
SLA	Agreement including commitments and consequences	A defined shortfall may qualify for service credits under agreed terms.

Targets require a clear scope, measurement window, and definition of failure. [R9]

Availability starts with a denominator

For a simplified time-based measure:

$$A = \frac{T - D}{T} \times 100\%, \qquad D_{\max} = T(1 - A_{\text{target}})$$

Here, T is eligible time and D is downtime, in the same units. Use the target as a fraction in the second formula.

For request-based success:

$$S = \frac{\text{successful eligible requests}}{\text{all eligible requests}} \times 100\%$$

These are different measurements. Do not substitute one for the other without agreement.

Worked example: the cost of another nine

Assume a 30-day month: $T = 30 \times 24 \times 60 = 43{,}200$ minutes.

Target	Allowed downtime / 30 days	Allowed downtime / 365 days
99%	432 min (7 h 12 min)	87.6 h
99.9%	43.2 min	8.76 h
99.99%	4.32 min	52.56 min

Steps: convert the period, calculate the failure fraction, then multiply.

Monthly values depend on month length. These figures assume the entire period is eligible.

Worked example: a reliability budget

Target: 99.9% availability over 30 days. Observed downtime: 30 minutes.

1. Allowed downtime: $43{,}200 \times 0.001 = 43.2$ minutes.
2. Observed availability: $100(1 - 30/43{,}200) = 99.9306\%$.
3. Remaining downtime budget: $43.2 - 30 = 13.2$ minutes.

The target is met for this example. A second 20-minute outage would exhaust the budget. A budget supports decisions; it is not planned downtime.

A useful service agreement is specific

- Specify service scope, success criteria, measurement source, and reporting window.
- Distinguish acknowledgement time, restoration time, and final resolution.
- Document maintenance exclusions and customer responsibilities.
- Read the actual remedy and claim process. Credits, refunds, or other consequences depend on the agreement.
- Teaching examples in this material are illustrative, not a contract template.

Monitoring asks answerable questions

- Are requests reaching the application? Measure traffic.
- Are users waiting? Measure latency distributions.
- Are operations failing? Measure errors and success criteria.
- Are resources becoming constrained? Measure CPU, memory, queueing, and connection use.
- Use metrics to spot a pattern, logs to inspect events, and traces to follow a request across services.

A small dashboard with a clear purpose

Signal	Example measurement	First interpretation
Latency	p95 request duration	Slow experience for some users
Traffic	Requests per second	Demand and completed work
Errors	Failed / eligible requests	Reliability under this workload
Saturation	CPU, queue length, pool usage	Resources nearing a limit
Memory	RSS and used heap	Process footprint and allocations

A high value is a clue. Compare it with workload, recent changes, and the user-facing symptom.

The average can hide a slow tail

Ten sorted response times (ms):

$$10,\ 10,\ 10,\ 10,\ 10,\ 10,\ 10,\ 10,\ 10,\ 1000$$

$$\text{mean} = 109 \text{ ms}, \qquad p50 = 10 \text{ ms}, \qquad p95 = 1000 \text{ ms}$$

Using nearest rank, $p95$ is item $\lceil 0.95n \rceil$.

Nine users have a fast response; one waits a second. Report the percentile convention and sample size. Ten observations are too few for a stable tail estimate.

Memory growth needs context

- RSS describes resident process memory. Used heap describes JavaScript heap allocations.
- A rising-and-falling heap can reflect normal allocation and garbage collection.
- A sustained rise after repeated comparable workloads may indicate retained objects. Compare after similar idle periods.
- Unbounded caches, growing arrays, timers, and event listeners are common investigation targets. A process restart hides the symptom temporarily.

Alerts should lead to an action

- Start with a user-facing symptom over a meaningful window.
- Example classroom alert: error fraction above 1% for five minutes, with enough requests to make the ratio useful.
- Add an owner, dashboard link, and first response steps.
- Avoid one alert per noisy event. Review false alarms and retention costs.
- Adaptive thresholds can help with patterns, but they also need validation.

Node.js: one event loop per ordinary process

- JavaScript callbacks normally run on one main event-loop thread.
- Node.js also uses operating-system facilities and a worker pool for some operations. Calling the whole runtime single-threaded is misleading.
- A long synchronous callback delays other callbacks in that process.
- Waiting asynchronously allows other work to progress; adding async to a function does not move CPU work off the event loop. [R2]

A cafe analogy for blocking

One cashier also cooks every order

A two-minute cooking task prevents the cashier from taking the next order. The queue grows even when the next customer only wants water.

A cashier delegates the cooking

The cashier records an order and serves the next customer while the kitchen works. Waiting overlaps, but kitchen capacity remains finite.

In the lab, a busy loop models blocking. A timer models waiting for I/O; it does not compute the same CPU task faster.

Optimisation is a controlled experiment

- State a hypothesis: repeated catalogue reads are spending time waiting for data.
- Capture a baseline with fixed request count and concurrency.
- Change one mechanism, such as a cache.
- Repeat the workload and compare latency, throughput, errors, and resource use.
- Keep the change only when the evidence and correctness checks support it.

Database performance begins with the query

- Fetch only the required columns and rows. Paginate large results.
- Inspect the query plan and representative execution timings.
- Indexes can accelerate matching reads but consume space and add write work.
- Bound connection pools. More application workers can multiply database connections.
- An ORM does not guarantee efficient SQL; inspect generated queries and repeated per-row requests. [R10]

Caching trades freshness for repeated work

1. Request a catalogue. Look for a valid cached result.
2. On a miss, read from the data source and cache the result.
3. On a hit, return the saved result without repeating the read.
4. After the time-to-live (TTL), refresh the value.

Lab choice: a single cached catalogue, five-second TTL, and one shared in-flight refresh per process.

Check freshness requirements, cache keys, invalidation, and memory bounds. Private user data must not leak through a shared cache key.

Choosing a cache location

Location	Good fit	Main complication
Process memory	Small repeated computations	Each worker has its own copy
Shared cache	Reused data across workers	Network dependency and invalidation
CDN / edge	Cacheable public assets	Cache headers and freshness

Caching helps repeated reads. It cannot fix every slow query or safely replace durable storage.

Vertical and horizontal scaling

Vertical: increase one machine's resources

- Often simpler to start.
- Limited by machine size and cost.
- One host can remain a failure domain.

Horizontal: add application instances

- Distribute traffic across instances.
- Requires coordination of shared state.
- Adds network and operational complexity.

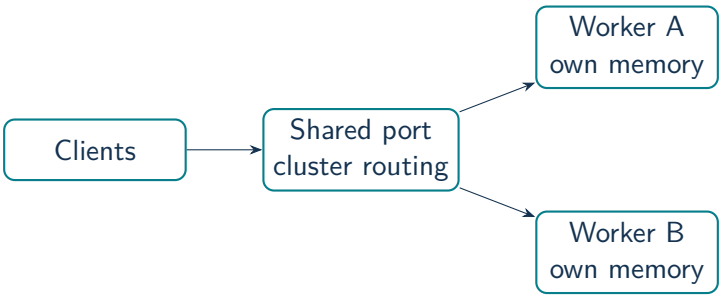
Extra instances help only if the constrained resource can scale with them.

Processes, threads, and load balancers

Mechanism	What it does	Typical use
Node.js cluster	Starts separate worker processes	More app capacity on one host
Worker threads	Runs JavaScript in other threads	CPU-heavy computation
Reverse proxy / LB	Routes traffic to app instances	Multiple processes or hosts

Cluster workers have separate memory. They are not threads inside one child process. A load balancer does not create application capacity by itself. [R3, R11]

Two workers still depend on one host



A failed worker can be replaced. A failed host stops both workers. Host-level resilience needs additional infrastructure.

Stateless workers simplify routing

- Each worker should be able to handle the next request using request data and shared services.
- Keep durable records in a database and shared sessions in a suitable store.
- A local variable, uploaded local file, or in-process cache is not automatically shared.
- JWTs can carry verifiable claims, but do not solve all state, revocation, or authorisation problems.
- Sticky sessions may help some workloads, but do not make local state durable.

Hands-on map: one small local service

Lab	Experiment	Evidence
1	Start the server and establish a baseline	Health and load output
2	Inspect metrics and planned errors	Counts and error fraction
3	Compare blocking with asynchronous waiting	Throughput and p95
4	Compare uncached and cached reads	Cold/warm latency
5	Start two workers and inspect state	PIDs and local counters
6	Optional PM2 process management	Status, reload, logs

Lab 1: a working baseline

```
cd lab
node --version
node app.js
```

Keep Terminal A running. In Terminal B, from the same folder:

```
node load.js http://127.0.0.1:3000/health 100 10
```

Expect 100 successful responses and zero errors. Record runtime version, concurrency, p95, and completed requests/second. See the companion guide for every step.

Lab 2: metrics and a controlled failure

```
node load.js http://127.0.0.1:3000/fail 10 1
```

Open `http://127.0.0.1:3000/metrics` in a browser.

- `/fail` intentionally returns HTTP 503.
- Counts belong to this process since startup.
- `/metrics` requests are excluded from these counts.
- Restart before an exact counting exercise.

Lab 3: blocking versus waiting

```
node load.js http://127.0.0.1:3000/blocking 100 10
node load.js http://127.0.0.1:3000/async 100 10
```

Both routes spend about 100 ms per isolated request. Under concurrency, asynchronous waits can overlap.

Prediction: the blocking route approaches about 10 requests/s in one worker. These two routes model different types of work. They do not demonstrate a CPU task becoming faster merely by using `await`.

Lab 4: cold and warm cache

```
node load.js http://127.0.0.1:3000/catalogue 20 1
node load.js http://127.0.0.1:3000/cached 1 1
node load.js http://127.0.0.1:3000/cached 20 1
```

Restart the server first for a cold cache. Run the final command within five seconds of the cold read.

Compare serial requests to make cache effects easy to see. Then wait more than five seconds and request again: the cache refresh should incur the simulated read delay.

Lab 5: two workers and separate counters

Stop `app.js` with Ctrl+C before starting the cluster.

```
node cluster.js
```

In Terminal B:

```
node inspect-workers.js
node load.js http://127.0.0.1:3000/blocking 100 10
```

Look for two PIDs and a counter sequence per PID. Distribution need not be exactly even. Compare against the single-worker baseline on the same machine.

Lab 6: optional PM2 workflow

Stop the manually started server or cluster. Install the optional package:

```
npm install --save-dev pm2
npx pm2 start app.js --name comp3028-lab -i 2
npx pm2 status
npx pm2 monit
```

Use Ctrl+C to leave the monitor; the managed application remains running. Start with two workers to keep the classroom experiment bounded. [R7, R8]

Reload, inspect, and clean up

```
npx pm2 reload comp3028-lab
npx pm2 logs comp3028-lab --lines 20
npx pm2 delete comp3028-lab
```

Compare PIDs before and after a reload. Verify health again.

Reload reduces interruption when the application supports it, but successful process replacement does not prove zero user errors. Persistence after a reboot requires separate setup. [R7, R8]

Interpreting a benchmark honestly

- Keep endpoint, total requests, concurrency, and hardware comparable.
- Run a warm-up, then at least three measured repetitions.
- Report each run and a summary such as the median. Do not hide failures.
- This client and server share a machine. The client can become a bottleneck.
- A small local benchmark does not establish production capacity or an SLA.

Worked interpretation: speed with reliability

Illustrative data, not measurements from your computer:

Run	Completed/s	p95 (ms)	Failed requests
Before	100	800	0/1000
After	160	500	40/1000

Throughput rises by 60%, and p95 falls by 37.5%.

But success is $960/1000 = 96\%$. If the target is 99.9%, this change fails the reliability requirement. Faster completed attempts do not necessarily mean more useful service.

A first incident response

- Confirm impact using a user journey, not only process status.
- Check the time window and recent deployments or configuration changes.
- Inspect errors, latency, and resource pressure.
- Mitigate with an appropriate rollback, load reduction, or recovery action.
- Record evidence and validate recovery, then investigate the cause and prevention.

Closing case: campus registration

At 09:05, catalogue p95 is 1.8 s, CPU is 25%, the database pool is saturated, and failure rate is 3%. The application has four workers.

1. Which evidence weakens the case for adding CPU?
2. What query and pool measurements would you request?
3. Which catalogue data can be cached, and for how long?
4. How would you validate the improvement?
5. Where should registration records and sessions live?

Prepare a two-minute recommendation with a measurable acceptance criterion.

Closing case: a reasoned response

- Investigate database contention and repeated reads first. Low application CPU does not rule out other bottlenecks.
- Inspect query duration, pool wait, active connections, and repeated lookups.
- Test a bounded catalogue cache if the freshness requirement allows it. Keep registrations durable and consistent.
- Repeat an equivalent workload and check correct registrations, errors, and latency.
- Add workers only if the measured bottleneck and downstream capacity justify it.

Group project: operational evidence

- Include the runtime version and commands needed to reproduce your experiment.
- Show baseline and revised measurements with comparable workloads.
- Explain one reliability target, one diagnosed bottleneck, and one trade-off.
- Keep meaningful commits, reviews, and issue evidence. Lines of code alone do not establish contribution quality.
- Follow the current LMS deadlines and marking instructions.

Assessment notes from the supplied lecture

- The source describes a contribution score with 25% peer review and 75% code contribution.
- Within its code-contribution component, the source describes 25% commits and 75% lines added to main.
- The source also mentions required peer reviews, final submission, and a Week 14 presentation.
- These are source-specific course notes, not newly confirmed policy. Check the current LMS for dates, access restrictions, and the final mark formula.

Key takeaways

- Define service quality before deciding what to optimise.
- Measure user-facing outcomes alongside CPU and memory.
- Use controlled experiments to distinguish blocking, I/O waiting, and repeated work.
- Keep shared state outside individual workers when scaling.
- A running process is only one part of a reliable service.

References

1. Original teaching material. Supplied COMP3028 Lecture 10: Optimising Server Management, undated PDF. Rewritten and expanded for beginners.
2. Node.js: event loop: `https://nodejs.org/en/learn/asynchronous-work/dont-block-the-event-loop`
3. Node.js: cluster: `https://nodejs.org/api/cluster.html`
4. Node.js: measurement: `https://nodejs.org/api/perf_hooks.html`
5. Node.js: process metrics: `https://nodejs.org/api/process.html`
6. Node.js: HTTP: `https://nodejs.org/api/http.html`

Official documentation consulted 10 November 2026. Labels R1–R11 follow this numbered list.

References (continued)

7. PM2: quick start: `https://pm2.keymetrics.io/docs/usage/quick-start/`
8. PM2: cluster mode: `https://pm2.keymetrics.io/docs/usage/cluster-mode/`
9. Google SRE: service-level objectives: `https://sre.google/sre-book/service-level-objectives/`
10. PostgreSQL: EXPLAIN: `https://www.postgresql.org/docs/current/using-explain.html`
11. NGINX: load balancing: `https://nginx.org/en/docs/http/load_balancing.html`

Official documentation consulted 10 November 2026. Labels R1–R11 follow this numbered list.